

THỰC HÀNH CI/CD GITOPS

HƯỚNG DẪN TRIỂN KHAI JENKINS - DOCKER - KUBERNETES - ARGO CD TRÊN AWS EC2

Mô hình 02 máy chủ Ubuntu 24.04 LTS | t3.small | 8 GB Disk

EC2-1: CI-Lab

GitHub Webhook
Jenkins
Docker
Elastic IP: 52.76.111.125



EC2-2: K8s-ArgoCD

K3s Kubernetes
Argo CD
NodePort App: 30080
Public IP động

Luồng hoàn chỉnh: Git push → Jenkins CI → Docker Hub → Jenkins cập nhật image tag → Argo CD Sync → Kubernetes Rolling Update

Instances (2/6) Info							
Last updated about 4 hours ago							
Connect Instance state Actions Launch instances							
Saved filter sets Choose filter set Find Instance by attribute or tag (case-sensitive)							
☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐	docker-demo...	i-0971b757d21a41b1f	Stopped	t3.small	-	ap-southeast-1a	-
☐	worker-node-1	i-09c0f81439768e717	Stopped	t3.small	-	ap-southeast-1a	-
☐	worker-node-2	i-03feb5d994d7a265c	Stopped	t3.small	-	ap-southeast-1a	-
☐	master-1	i-0bd4da3ff5efb3e07	Stopped	t3.small	-	ap-southeast-1a	-
☒	CICD_Lab	i-006093f0868c8845c	Running	t3.small	3/3 checks pass	ap-southeast-1a	ec2-52-76-111-125
☒	K8s-ArgoCD	i-0727f3c5bc88bf1f6	Running	t3.small	3/3 checks pass	ap-southeast-1a	ec2-47-129-158-13

2 instances selected

Monitoring

Configure CloudWatch agent

Alarm recommendations

Investigate with AI - new

3h 1d 1w 1h

UTC timezone

Explore related

Hình minh họa: Hai EC2 đang Running trên AWS Console

Người thực hiện: Kim Việt Anh

MỤC LỤC

1. Tổng quan kiến trúc và thông số hệ thống
2. Chuẩn bị AWS EC2 và Security Group
3. Cài đặt máy CI-Lab: Docker và Jenkins
4. Chuẩn bị mã nguồn GitHub và Docker Hub
5. Cấu hình Jenkins Pipeline và GitHub Webhook
6. Cài đặt máy K8s-ArgoCD: Docker, K3s và Argo CD
7. Tạo Kubernetes manifests và cấu hình Argo CD Application
8. Kiểm thử toàn bộ luồng CI/CD GitOps
9. Dọn dẹp, vận hành và xử lý lỗi thường gặp
10. Phụ lục: File cấu hình hoàn chỉnh và bảng lệnh tra cứu

☐ Quy ước trong tài liệu

Mỗi khối lệnh đều ghi rõ chạy trên máy nào và trong thư mục nào. Các khung màu đỏ “ẢNH CẦN CHỤP” là vị trí nên bổ sung ảnh minh họa khi hoàn thiện báo cáo.

1. TỔNG QUAN KIẾN TRÚC VÀ THÔNG SỐ HỆ THỐNG

Tài liệu mô tả quy trình triển khai một hệ thống CI/CD theo mô hình GitOps trên AWS EC2. Jenkins chịu trách nhiệm CI: checkout, cài dependencies, test, build và push Docker image. Argo CD chịu trách nhiệm CD: theo dõi thư mục k8s trong GitHub, tự động đồng bộ thay đổi vào Kubernetes và thực hiện Rolling Update.

1.1. Thông số hai máy chủ

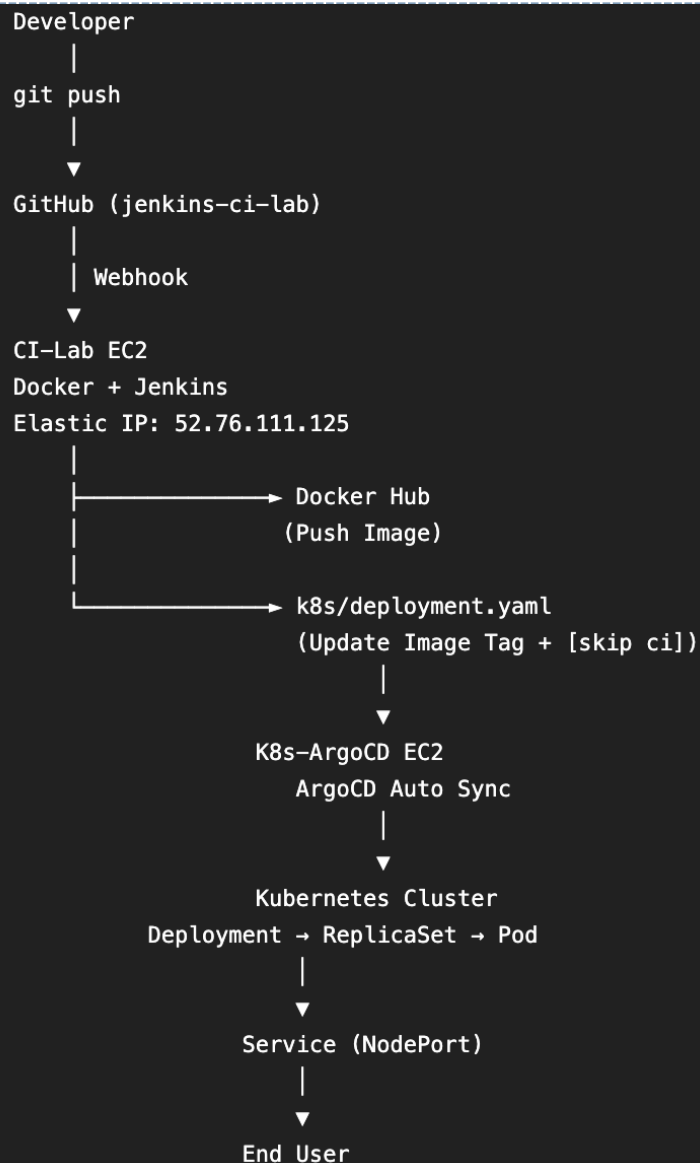
Thành phần	EC2-1: CI-Lab	EC2-2: K8s-ArgoCD
Vai trò	CI Server	CD Server
Hệ điều hành	Ubuntu Server 24.04 LTS	Ubuntu Server 24.04 LTS
Instance type	t3.small (2 vCPU, 2 GiB RAM)	t3.small (2 vCPU, 2 GiB RAM)
Ổ đĩa	8 GB gp3	8 GB gp3
IP Public	Elastic IP 52.76.111.125	IP động; ví dụ hiện tại 47.129.158.130
Phần mềm	Docker, Jenkins, Git, Docker CLI	Docker, K3s Kubernetes, kubectl, Argo CD
Cổng chính	22, 8080, 50000	22, NodePort Argo CD, 30080

1.2. Luồng xử lý end-to-end

```
Developer → git push origin main
↓
GitHub Webhook → Jenkins
↓
Install → Test → docker build → docker push
↓
Jenkins sửa k8s/deployment.yaml bằng BUILD_NUMBER
↓
Jenkins git push commit [skip ci]
↓
Argo CD phát hiện manifest mới
↓
Kubernetes tạo ReplicaSet/Pod mới → Service NodePort 30080
```

❑ Vì sao CI-Lab cần Elastic IP?

GitHub Webhook gọi trực tiếp URL Jenkins `http://52.76.111.125:8080/github-webhook/`. Nếu IP thay đổi sau khi Stop/Start EC2, webhook sẽ lỗi. Elastic IP giúp địa chỉ Jenkins luôn cố định.



Sơ đồ tổng thể mô hình CI/CD GitHub → Jenkins → Docker Hub → GitHub k8s → Argo CD → Kubernetes.

2. CHUẨN BỊ AWS EC2 VÀ SECURITY GROUP

2.1. Tạo máy CI-Lab

1. AWS Console → EC2 → Instances → Launch instances.
2. Name: CI-Lab; AMI: Ubuntu Server 24.04 LTS; Instance type: t3.small.
3. Key pair: chọn cùng một file .pem dùng cho cả hai máy.
4. Storage: 8 GiB gp3. Network: dùng cùng VPC/Subnet để dễ quản lý.
5. Launch instance và đợi trạng thái 3/3 checks passed.

Instance summary for i-006093f0868c8845c (CI-CD_Lab) [Info](#)

Updated less than a minute ago

[Refresh](#)
[Connect](#)
[Instance state ▼](#)
[Actions ▼](#)

Instance ID i-006093f0868c8845c	Public IPv4 address 52.76.111.125 open address ↗	Private IPv4 addresses 172.31.36.152
IPv6 address -	Instance state Running	Public DNS ec2-52-76-111-125.ap-southeast-1.compute.amazonaws.com open address ↗
Hostname type IP name: ip-172-31-36-152.ap-southeast-1.compute.internal	Private IP DNS name (IPv4 only) ip-172-31-36-152.ap-southeast-1.compute.internal	Elastic IP addresses 52.76.111.125 [Public IP]
Answer private resource DNS name IPv4 (A)	Instance type t3.small	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more ↗
Auto-assigned IP address -	VPC ID vpc-0ca18cf7ac2912b62 ↗	Auto Scaling Group name -
IAM role -	Subnet ID subnet-0334144985ed74b4a ↗	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:ap-southeast-1:042134552922:instance/i-006093f0868c8845c	

Cấu hình Launch Instance của CI-Lab

2.2. Gán Elastic IP cho CI-Lab

- EC2 → Network & Security → Elastic IP addresses → Allocate Elastic IP address.
- Chọn Elastic IP vừa tạo → Actions → Associate Elastic IP address.
- Resource type: Instance → chọn CI-Lab → Associate.

Sau khi gán, IP cố định của CI-Lab là: 52.76.111.125.

52.76.111.125

Actions Associate Elastic IP address

Summary

Allocated IPv4 address

52.76.111.125

Association ID

eipassoc-0cbcaa2b21b825a40

Network interface ID

eni-081ed173d2d532209

Address pool

Amazon

Type

Public IP

Scope

VPC

Network interface owner account ID

042134552922

Network border group

ap-southeast-1

Allocation ID

eipalloc-0303d248bd22b29f5

Associated instance ID

i-006093f0868c8845c

Public DNS

ec2-52-76-111-125.ap-southeast-1.compute.amazonaws.com

Service managed

-

Reverse DNS record

-

Private IP address

172.31.36.152

NAT Gateway ID

-

Tags(0)

Manage tags

Key

Value

No tags associated with this resource

Click the Manage tags button to add your first tag

Manage tags

Elastic IP đã Associate CI-Lab.

2.3. Tạo máy K8s-ArgoCD

- Tạo EC2 tương tự CI-Lab: Ubuntu 24.04, t3.small, 8 GB disk, cùng key pair, cùng security group.
- Name: K8s-ArgoCD. Máy này có thể dùng Public IP động để tiết kiệm chi phí Elastic IP.

IP động của CD Server

Ví dụ hiện tại là 47.129.158.130. Sau mỗi lần Stop/Start, cần kiểm tra lại Public IPv4 và cập nhật URL truy cập Argo CD hoặc ứng dụng.

2.4. Security Group khuyến nghị

Máy	Port	Mục đích	Source khuyến nghị
CI-Lab	22/TCP	SSH	My IP
CI-Lab	8080/TCP	Jenkins Web UI/Webhook	0.0.0.0/0 cho lab
CI-Lab	50000/TCP	Jenkins agent inbound	Chỉ mở khi dùng agent
K8s-ArgoCD	22/TCP	SSH	My IP
K8s-ArgoCD	30000-32767/TCP	NodePort (có thể mở riêng từng cổng)	0.0.0.0/0 cho lab
K8s-ArgoCD	30080/TCP	Ứng dụng Node.js	0.0.0.0/0
K8s-ArgoCD	NodePort Argo CD	Argo CD Web UI	0.0.0.0/0

Bảo mật

Trong môi trường thật, không mở 0.0.0.0/0 cho SSH/Jenkins/Argo CD. Nên giới hạn IP nguồn, dùng VPN, reverse proxy HTTPS hoặc Load Balancer. Nhưng đây là bài lab, nên tôi sẽ vẫn ưu tiên để 0.0.0.0/0 và tôi tận dụng Security Group cũ của tôi từ trước

Details

Security group name
vanh-docker

Security group ID
sg-0cf5d0fb529084e8e

Description
Allows SSH

VPC ID
vpc-0ca18cf7ac2912b62

Owner
042134552922

Inbound rules count
10 Permission entries

Outbound rules count
1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (10)



Manage tags

Edit inbound rules

Search

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-0060af9bd2fcf790e	IPv4	Custom TCP	TCP	30080
☐	-	sgr-00ec3c24025b654a2	IPv4	Custom TCP	TCP	8080
☐	-	sgr-0f886da032f4acac6	IPv4	Custom TCP	TCP	8083
☐	-	sgr-0e3623d02f9952014	IPv4	Custom TCP	TCP	8081
☐	-	sgr-0296bcd750ea91e6	IPv4	SSH	TCP	22
☐	-	sgr-03c1f7c2bde0cb66a	IPv4	Custom TCP	TCP	31140
☐	-	sgr-05d096d8cc7462260	IPv4	Custom TCP	TCP	8082
☐	-	sgr-0fbc64f63e4368098	IPv4	Custom TCP	TCP	8084
☐	-	sgr-0665568a271f9c868	IPv4	Custom TCP	TCP	3000
☐	-	sgr-0df0279454ad6996a	IPv4	Custom TCP	TCP	30402

Inbound rules của Security Group.

3. CÀI ĐẶT MÁY CI-LAB: DOCKER VÀ JENKINS

3.1. SSH vào CI-Lab

MÁY / VỊ TRÍ Máy cá nhân | Thư mục: Thư mục chứa file .pem

```
chmod 400 key-cua-ban.pem  
ssh -i key-cua-ban.pem ubuntu@52.76.111.125
```

Ý nghĩa: Phân quyền riêng tư cho private key và kết nối SSH vào CI-Lab bằng user ubuntu.



SSH thành công vào CI-Lab.

3.2. Cập nhật hệ thống và đặt hostname

MÁY / VỊ TRÍ CI-Lab | Thư mục: /home/ubuntu

```
sudo apt update && sudo apt upgrade -y
sudo hostnamectl set-hostname ci-server
exec bash
```

□ **Ý nghĩa:** Cập nhật package và đổi hostname để dễ phân biệt với CD Server.

3.3. Cài Docker Engine từ repository chính thức

MÁY / VỊ TRÍ CI-Lab | Thư mục: /home/ubuntu

```
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

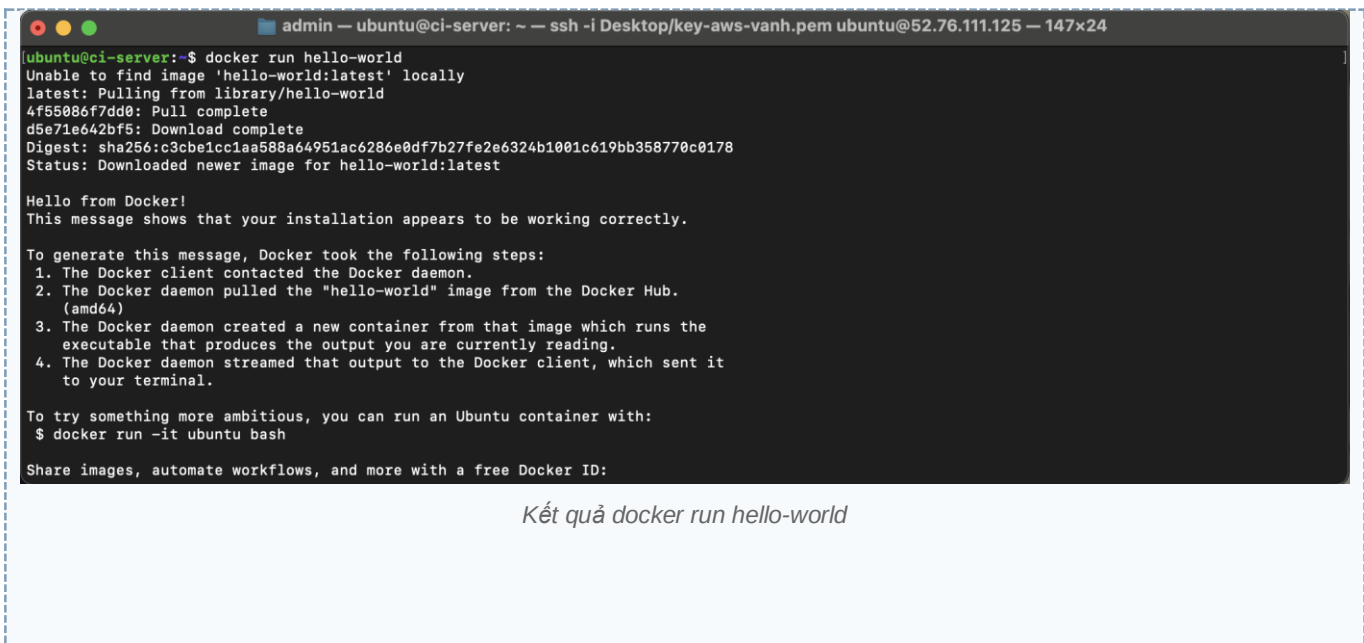
□ **Ý nghĩa:** Thêm GPG key và repository Docker, sau đó cài Docker Engine, Docker CLI, containerd, Buildx và Compose v2.

3.4. Cho user ubuntu chạy Docker không cần sudo

MÁY / VỊ TRÍ CI-Lab | Thư mục: /home/ubuntu

```
sudo usermod -aG docker $USER
newgrp docker
docker run hello-world
```

□ **Ý nghĩa:** Thêm user ubuntu vào group docker và chạy container kiểm thử.



3.5. Tạo Docker Compose cho Jenkins

MÁY / VỊ TRÍ CI-Lab | Thư mục: /home/ubuntu/jenkins-ci-lab

```
mkdir -p ~/jenkins-ci-lab
cd ~/jenkins-ci-lab
nano docker-compose.yml
```

Nội dung docker-compose.yml:

```
services:
  jenkins:
    image: jenkins/jenkins:lts-jdk17
    container_name: jenkins
    user: root
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    restart: unless-stopped

volumes:
  jenkins_home:
```

□ **Ý nghĩa:** Chạy Jenkins bằng container, lưu dữ liệu trong named volume và mount Docker socket để Jenkins điều khiển Docker Engine của host.

3.6. Khởi chạy Jenkins

MÁY / VỊ TRÍ CI-Lab | Thư mục: /home/ubuntu/jenkins-ci-lab

```
docker compose up -d
docker compose ps
docker logs -f jenkins
```

□ **Ý nghĩa:** Tạo container Jenkins ở chế độ nền, kiểm tra trạng thái và xem log khởi động.

```
admin — ubuntu@ci-server: ~/jenkins-ci-lab — ssh -i Desktop/key-aws-vanh.pem ubuntu@52.76.111.125 — 147x24
ubuntu@ci-server:~/jenkins-ci-lab$ docker compose up -d
[+] Running 1/1
  ✓ Container jenkins Running
ubuntu@ci-server:~/jenkins-ci-lab$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
jenkins             jenkins/jenkins:lts-jdk17  "/usr/bin/tini -- /u..."  jenkins    3 days ago     Up 6 hours    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
ubuntu@ci-server:~/jenkins-ci-lab$ docker logs -f jenkins
Running from: /usr/share/jenkins/jenkins.war
webroot: /var/jenkins_home/war
2026-07-31 02:52:10.430+0000 [id=1] INFO winstone.Logger#logInternal: Beginning extraction from war file
2026-07-31 02:52:11.478+0000 [id=1] WARNING o.e.j.ee9.nested.ContextHandler#setContextPath: Empty contextPath
2026-07-31 02:52:11.591+0000 [id=1] INFO org.eclipse.jetty.server.Server#doStart: jetty-12.1.5; built: 2025-12-03T22:18:24.732Z; git: 4905d0
9d7b6801e792d7b73946cf7b66e4be25d6; jvm 17.0.18+8
2026-07-31 02:52:12.381+0000 [id=1] INFO o.e.j.e.w.StandardDescriptorProcessor#visitServlet: NO JSP Support for /, did not find org.eclipse.
jetty.ee9.jsp.JettyJspServlet
2026-07-31 02:52:12.499+0000 [id=1] INFO o.e.j.s.DefaultSessionIdManager#doStart: Session workerName=node0
2026-07-31 02:52:13.176+0000 [id=1] INFO hudson.WebAppMain#contextInitialized: Jenkins home directory: /var/jenkins_home found at: EnvVars.m
asterEnvVars.get("JENKINS_HOME")
2026-07-31 02:52:13.384+0000 [id=1] INFO o.e.j.s.handler.ContextHandler#doStart: Started oeje9n.ContextHandler$CoreContextHandler@68577ba8{J
enkins v2.541.3,,b=file:///var/jenkins_home/war/,a=AVAILABLE,h=oeje9n.ContextHandler$CoreContextHandler$CoreToNestedHandler@1108adc8{STARTED}}
2026-07-31 02:52:13.411+0000 [id=1] INFO o.e.j.server.AbstractConnector#doStart: Started oejs.ServerConnector@4b311ff8{HTTP/1.1, (http/1.1)}
{0.0.0.0:8080}
2026-07-31 02:52:13.414+0000 [id=1] INFO org.eclipse.jetty.server.Server#doStart: Started oejs.Server@7a34f66a{STARTING}[12.1.5,sto=0] @1826
```

Docker compose ps và container Jenkins Up

3.7. Cài Docker CLI bên trong Jenkins container

MÁY / VỊ TRÍ CI-Lab | Thư mục: /home/ubuntu/jenkins-ci-lab

```
docker exec -u root -it jenkins bash
```

MÁY / VỊ TRÍ Bên trong container Jenkins | Thư mục: /

```
apt-get update
apt-get install -y ca-certificates curl gnupg lsb-release
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/debian $(. /etc/os-release && echo "$VERSION_CODENAME") stable" >
/etc/apt/sources.list.d/docker.list
apt-get update
apt-get install -y docker-ce-cli docker-buildx-plugin
docker version
exit
```

☐ Ý nghĩa: Jenkinsfile gọi lệnh docker, vì vậy container Jenkins phải có Docker CLI. Docker daemon vẫn là daemon trên host thông qua /var/run/docker.sock.

☐ Khuyến nghị lâu dài

Nếu thường xuyên recreate container Jenkins, nên tạo custom Jenkins image có sẵn Docker CLI. Trong lab này, bước cài trực tiếp đủ để thực hành.

3.8. Thiết lập Jenkins Web UI

MÁY / VỊ TRÍ CI-Lab | Thư mục: /home/ubuntu/jenkins-ci-lab

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

☐ Ý nghĩa: Lấy mật khẩu administrator ban đầu của Jenkins.

11. Mở http://52.76.111.125:8080.

12. Dán initialAdminPassword → Continue.
13. Chọn Install suggested plugins.
14. Tạo tài khoản admin và hoàn tất Setup Wizard.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

`/var/jenkins_home/secrets/initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

Continue

Getting Started

Getting Started

☒ Folders	☒ OWASP Markup Formatter	☐ Build Timeout	☐ Credentials Binding	** commons-lang3 v3.x Jenkins API ** Jenkins API
☐ Timestampers	☐ Workspace Cleanup	☐ Ant	☐ Gradle	Folders OWASP Markup Formatter ** Jenkins API
☐ Pipeline	☐ Github Branch Source	☐ Pipeline: Github Groovy Libraries	☐ Pipeline Graph View	
☐ Git	☐ SSH Build Agents	☐ Matrix Authorization Strategy	☐ LDAP	
☐ Email Extension	☐ Mailer	☐ Dark Theme		

Jenkins 2.541.3

 Jenkins

+ New Item

 Build History

Build Queue

No builds in the queue.

Build Executor Status

0/2

Search, Settings, Help icons

Add description

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job

Set up a distributed build

Set up an agent

Configure a cloud

Learn more about distributed builds

Ânh: Unlock Jenkins, Customize Jenkins, Dashboard.

4. CHUẨN BỊ MÃ NGUỒN GITHUB VÀ DOCKER HUB

4.1. Cấu trúc repository một repo

```
jenkins-ci-lab/  
├─ app.js  
├─ package.json  
├─ Dockerfile  
├─ Jenkinsfile  
├─ k8s/  
│  └─ deployment.yaml  
│  └─ service.yaml
```

Repo GitHub sử dụng: <https://github.com/anhkv100297/jenkins-ci-lab>

4.2. File ứng dụng Node.js

app.js dùng Express, lắng nghe cổng 3000 và trả về giao diện trạng thái CI/CD. package.json khai báo script start và test.

```
{  
  "name": "sample-ci-app",  
  "version": "1.0.0",  
  "scripts": {  
    "start": "node app.js",  
    "test": "node -e \"console.log('Tests passed')\""  
  },  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```

4.3. Dockerfile

```
FROM node:20-alpine  
  
WORKDIR /app  
COPY package*.json ./  
RUN npm install  
COPY . .  
EXPOSE 3000  
CMD ["npm", "start"]
```

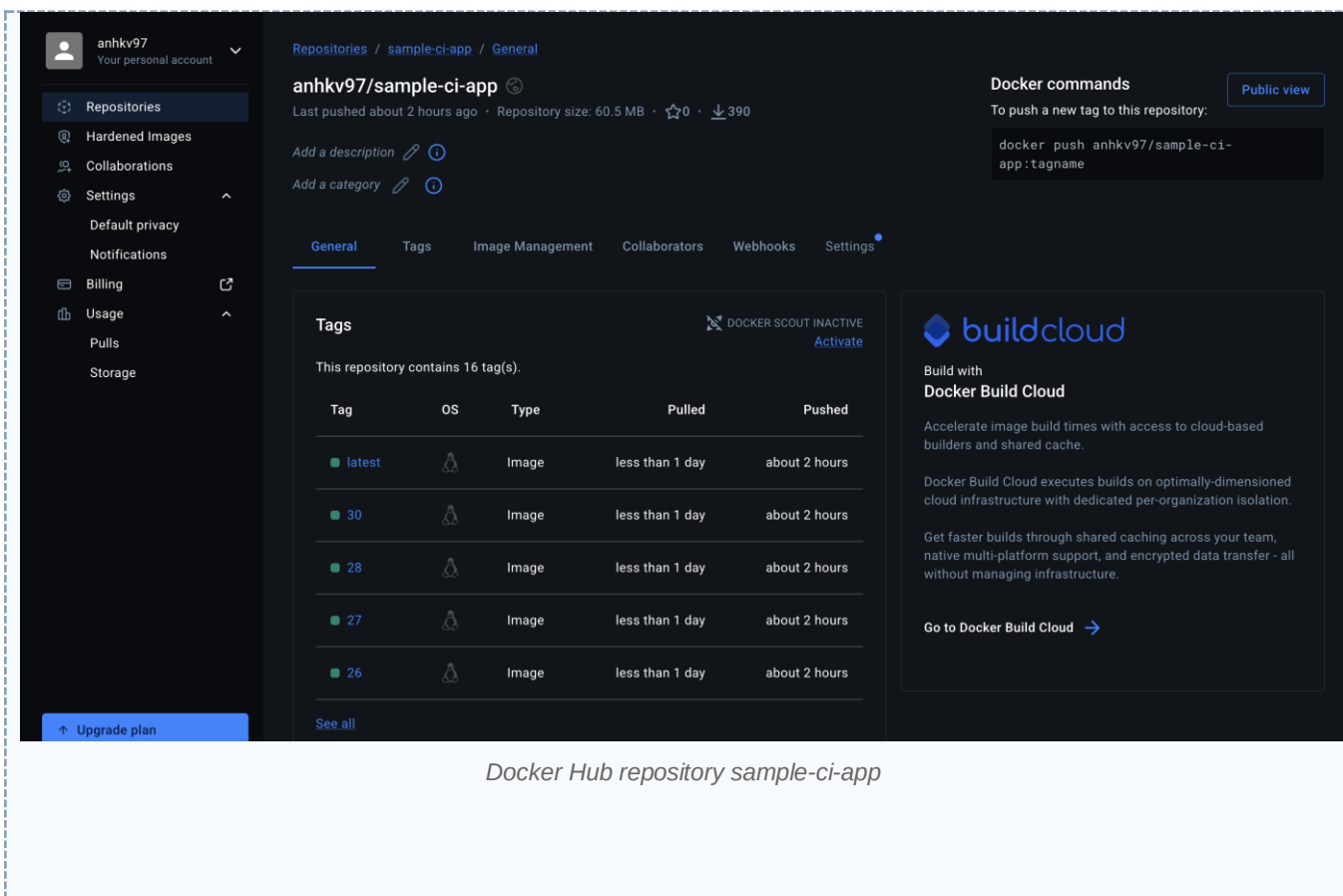
□ **Ý nghĩa:** Tạo image Node.js nhỏ gọn bằng Alpine, cài dependencies, copy source và chạy npm start.

4.4. Tạo Docker Hub repository

15. Đăng nhập Docker Hub bằng username anhkv97.

16. Repositories → Create repository → Name: sample-ci-app → Visibility: Public.

Tên image đầy đủ: anhkv97/sample-ci-app



Docker Hub repository sample-ci-app

4.5. Tạo Docker Hub Personal Access Token

17. Docker Hub → Account settings → Personal access tokens → Generate new token.
18. Tên token: jenkins-ci; quyền: Read & Write.
19. Copy token ngay khi Docker Hub hiển thị; token sẽ dùng làm password trong Jenkins Credentials.

❑ Không ghi token vào Jenkinsfile

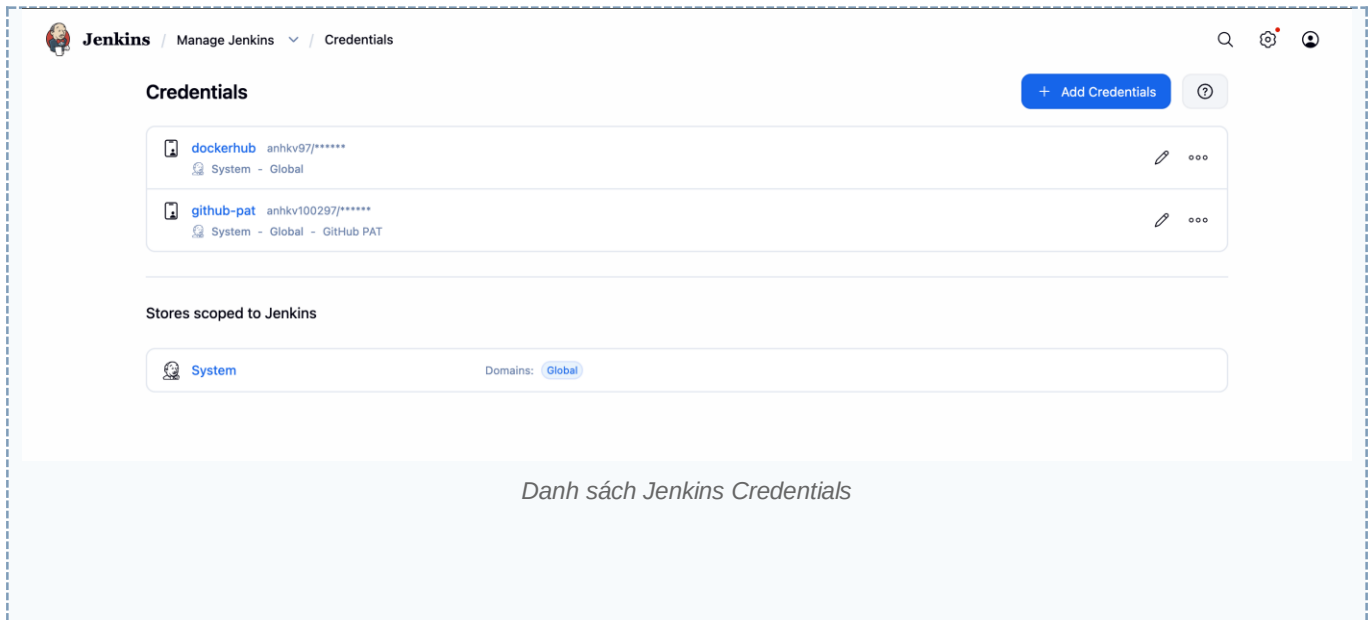
Luôn lưu token trong Jenkins Credentials để Jenkins che giá trị trong Console Output.

5. CẤU HÌNH JENKINS PIPELINE VÀ GITHUB WEBHOOK

5.1. Tạo Jenkins Credentials

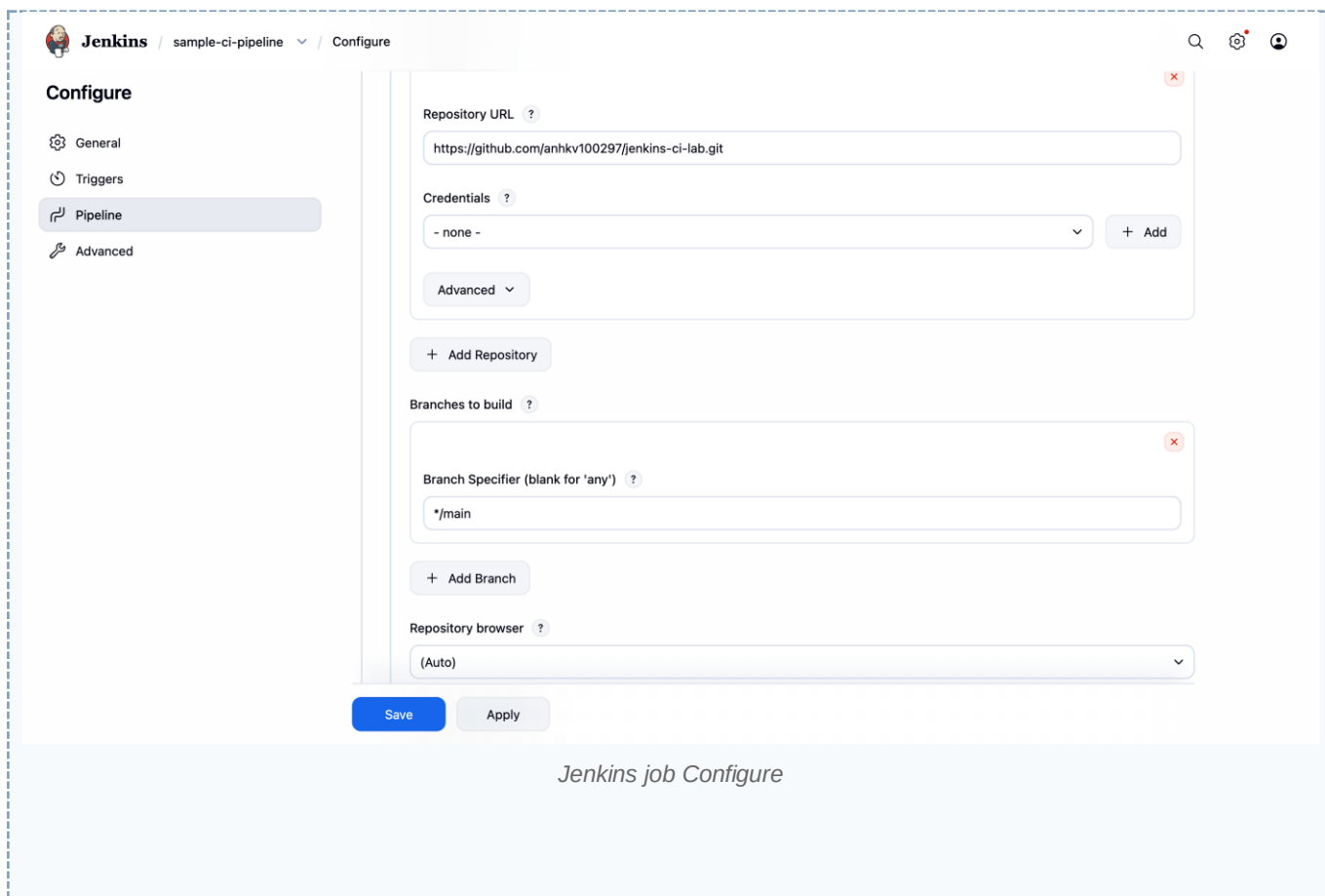
20. Manage Jenkins → Credentials → System → Global credentials → Add Credentials.

ID	Kind	Username	Password/Secret
dockerhub	Username with password	anhkv97	Docker Hub PAT
github-pat	Username with password	anhkv100297	GitHub Personal Access Token có quyền ghi repo



5.2. Tạo Pipeline job

21. Jenkins Dashboard → New Item → tên sample-ci-pipeline → Pipeline.
22. Build Triggers: chọn GitHub hook trigger for GITScm polling.
23. Pipeline Definition: Pipeline script from SCM.
24. SCM: Git; Repository URL: <https://github.com/anhkv100297/jenkins-ci-lab.git>; Branch: */main; Script Path: Jenkinsfile.
25. Save.



5.3. Jenkinsfile hoàn chỉnh chống vòng lặp

Jenkins build source commit bình thường. Sau khi cập nhật image tag trong deployment.yaml, Jenkins tạo commit có [skip ci]. Webhook vẫn gọi build tiếp theo, nhưng build đó thoát ngay và hiển thị “Skipped GitOps manifest commit”.

```

pipeline {
    agent any

    options {
        disableConcurrentBuilds()
    }

    environment {
        DOCKER_IMAGE = "anhkv97/sample-ci-app"
        DEPLOYMENT_FILE = "k8s/deployment.yaml"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('CI/CD Pipeline') {
            steps {
                script {
                    def commitMessage = sh(
                        script: 'git log -1 --pretty=%B',
                        returnStdout: true
                    ).trim()

                    def changedFiles = sh(
                        script: 'git show --pretty="" --name-only HEAD',
                        returnStdout: true
                    ).trim()

                    echo "Commit message: ${commitMessage}"
                    echo "Changed files: ${changedFiles}"

                    if (commitMessage.contains('[skip ci]')) {
                        currentBuild.description = "Skipped GitOps manifest commit"
                        echo "Phát hiện [skip ci]. Dừng pipeline để tránh vòng lặp."
                        return
                    }

                    def files = changedFiles.split('\n')
                        .collect { it.trim() }
                        .findAll { it }

                    def onlyK8sChanged = !files.isEmpty() &&
                        files.every { it.startsWith('k8s/') }

                    if (onlyK8sChanged) {
                        currentBuild.description = "Skipped k8s-only commit"
                        echo "Commit chỉ thay đổi thư mục k8s/. Dừng pipeline."
                        return
                    }

                    stage('Install dependencies') {
                        sh '''
                            docker run --rm \
                                --volumes-from "$HOSTNAME" \
                                -w "$WORKSPACE" \
                                node:20-alpine npm install
                        '''
                    }

                    stage('Test') {
                        sh '''
                            docker run --rm \
                                --volumes-from "$HOSTNAME" \
                                -w "$WORKSPACE" \
                                node:20-alpine npm test
                        '''
                    }
                }
            }
        }
    }
}

```

```

        node:20-alpine npm test
    ...
}

stage('Build Docker image') {
    sh '''
        docker build \
        -t ${DOCKER_IMAGE}:${BUILD_NUMBER} \
        -t ${DOCKER_IMAGE}:latest .
    ...
}

stage('Push Docker image') {
    withCredentials([usernamePassword(
        credentialsId: 'dockerhub',
        usernameVariable: 'DOCKERHUB_USERNAME',
        passwordVariable: 'DOCKERHUB_TOKEN'
    )]) {
        sh '''
            echo "${DOCKERHUB_TOKEN}" | docker login \
            -u "${DOCKERHUB_USERNAME}" --password-stdin
            docker push ${DOCKER_IMAGE}:${BUILD_NUMBER}
            docker push ${DOCKER_IMAGE}:latest
            docker logout
        ...
    }
}

stage('Update Kubernetes manifest') {
    sh '''
        sed -i \
        "s|image: ${DOCKER_IMAGE}:.*|image: ${DOCKER_IMAGE}:${BUILD_NUMBER}|" \
        ${DEPLOYMENT_FILE}
        grep "image:" ${DEPLOYMENT_FILE}
    ...
}

stage('Push manifest to GitHub') {
    withCredentials([usernamePassword(
        credentialsId: 'github-pat',
        usernameVariable: 'GITHUB_USERNAME',
        passwordVariable: 'GITHUB_TOKEN'
    )]) {
        sh '''
            git config user.name "Jenkins CI"
            git config user.email "jenkins@example.local"
            git add ${DEPLOYMENT_FILE}

            if git diff --cached --quiet; then
                echo "Manifest không thay đổi."
            else
                git commit -m "Update image to ${DOCKER_IMAGE}:${BUILD_NUMBER} [skip ci]"
                git push \

"https://${GITHUB_USERNAME}:${GITHUB_TOKEN}@github.com/anhkv100297/jenkins-ci-lab.git" \
                HEAD:main
            fi
        ...
    }
}

}

}

}

}

post {
    success {
        echo "Pipeline hoàn thành hoặc được bỏ qua an toàn."
    }
}

```

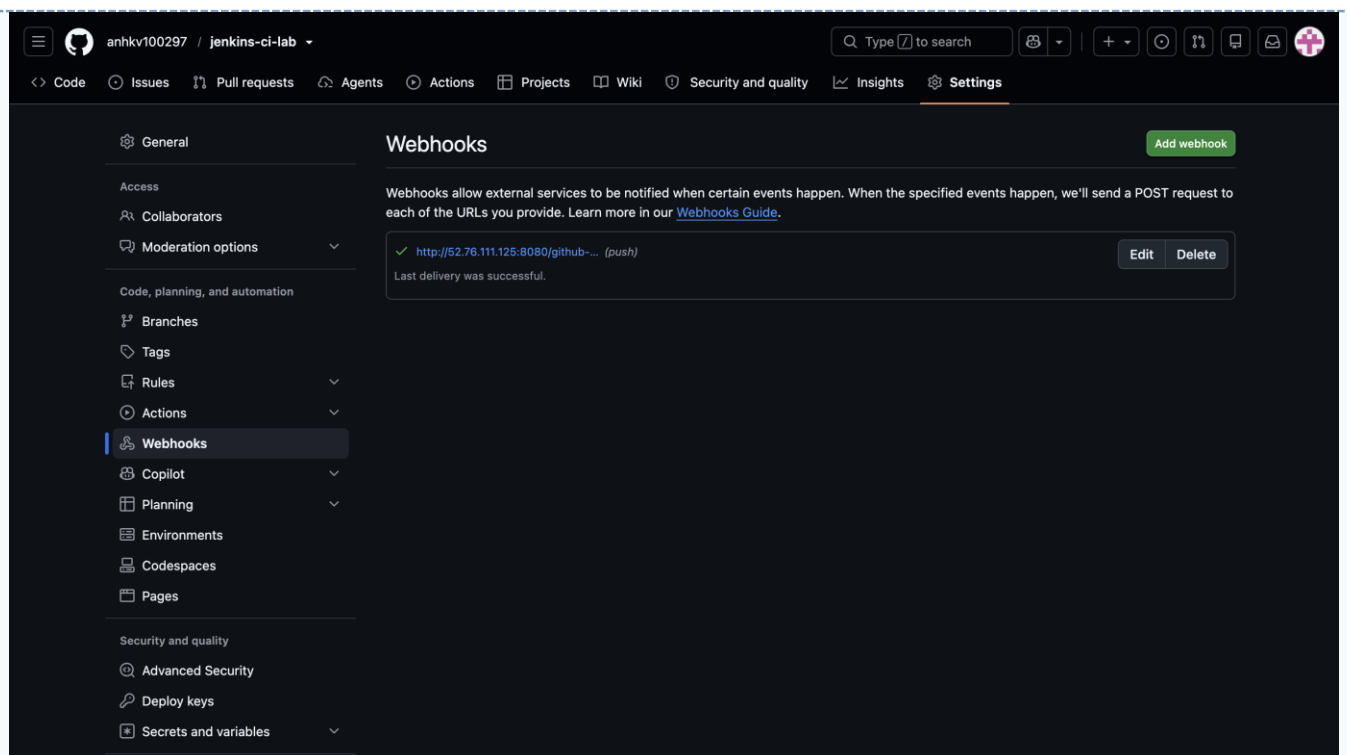
```

    }
    failure {
        echo "Pipeline thất bại. Kiểm tra Console Output."
    }
    always {
        sh 'docker image prune -f || true'
    }
}
}

```

5.4. Tạo GitHub Webhook

26. GitHub repo jenkins-ci-lab → Settings → Webhooks → Add webhook.
27. Payload URL: <http://52.76.111.125:8080/github-webhook/>
28. Content type: application/json.
29. Which events: Just the push event; Active: bật.
30. Add webhook. Kiểm tra dấu tick xanh và “Last delivery was successful”.



Github Webhook thành công

□ Hiện tượng build hai lần là đúng

Build thứ nhất chạy đầy đủ và push manifest. Build thứ hai do webhook của commit [skip ci] nhưng Jenkins dừng ngay. Không có build image hay Pod mới ở lần thứ hai.

6. CÀI ĐẶT MÁY K8S-ARGOCD: DOCKER, K3S VÀ ARGO CD

6.1. SSH, cập nhật và đặt hostname

MÁY / VỊ TRÍ

Máy cá nhân | Thư mục: Thư mục chứa file .pem

```
ssh -i key-cua-ban.pem ubuntu@47.129.158.130
```

MÁY / VỊ TRÍ

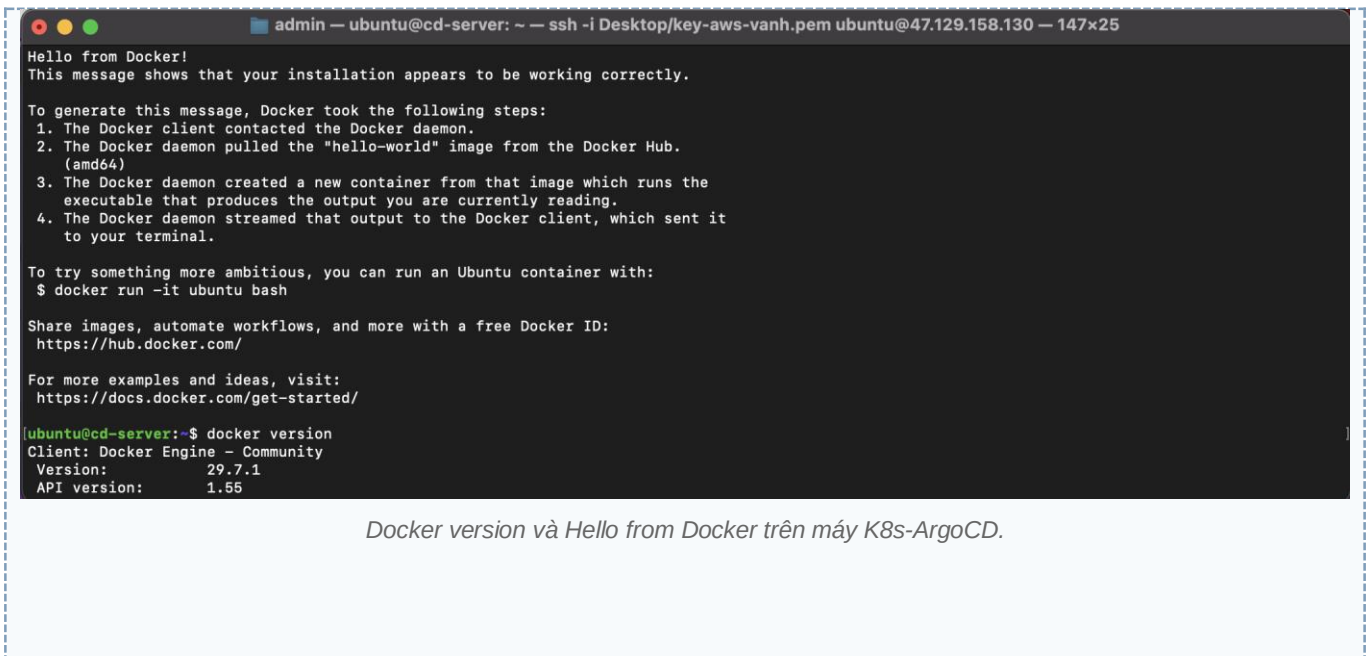
K8s-ArgoCD | Thư mục: /home/ubuntu

```
sudo apt update && sudo apt upgrade -y
sudo hostnamectl set-hostname cd-server
exec bash
```

6.2. Cài Docker trên CD Server

Thực hiện lại các bước cài Docker giống mục 3.3 và thêm user ubuntu vào group docker. Docker dùng để kiểm tra/troubleshoot; K3s sử dụng containerd tích hợp.

```
docker run hello-world
docker version
```



6.3. Cài K3s Kubernetes

MÁY / VỊ TRÍ

K8s-ArgoCD | Thư mục: /home/ubuntu

```
curl -sLf https://get.k3s.io | sh -
```

□ **Ý nghĩa:** Tải và cài K3s server, containerd, kubelet, API Server, CoreDNS, Traefik và local-path storage.

```
sudo systemctl status k3s --no-pager
sudo kubectl get nodes -o wide
sudo kubectl get pods -A
```

□ **Ý nghĩa:** Kiểm tra service K3s, node Ready và các Pod hệ thống.

```

ubuntu@cd-server:~$ sudo kubectl get nodes -o wide
NAME              STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
cd-server-2.3.2-k3s2 Ready    control-plane 17h   v1.36.2+k3s1 172.31.45.152 <none>        Ubuntu 24.04.4 LTS   7.0.0-1009-aws (amd64) containerd://
ubuntu@cd-server:~$ sudo kubectl get pods -A
NAMESPACE   NAME                                                                 READY   STATUS    RESTARTS   AGE
argocd      argocd-application-controller-0                                     1/1     Running   1 (16h ago)  17h
argocd      argocd-applicationset-controller-85d4d9d4d-4lqv9                   0/1     CrashLoopBackOff 70 (109s ago)  17h
argocd      argocd-dex-server-5bf99699b5-nd2hs                                1/1     Running    1 (16h ago)  17h
argocd      argocd-notifications-controller-5bf6c68f6d-zxsdv                   1/1     Running    1 (16h ago)  17h
argocd      argocd-redis-6548db7b7c-6psp2                                     1/1     Running    1 (16h ago)  17h
argocd      argocd-repo-server-697876794c-wrll2                               1/1     Running    1 (16h ago)  17h
argocd      argocd-server-699676944-58hb7                                     1/1     Running    1 (16h ago)  17h
default     sample-ci-app-85b74d645f-cfrnb                                    1/1     Running    0              129m
kube-system coredns-5f5694d56b-dsb9f                                         1/1     Running    1 (16h ago)  17h
kube-system helm-install-traefik-88qk2                                0/1     Completed   2 (17h ago)  17h
kube-system helm-install-traefik-crd-5145f                          0/1     Completed   0              17h
kube-system local-path-provisioner-58d557dc48-vzk6t                    1/1     Running    1 (16h ago)  17h
kube-system metrics-server-7c86f97b8d-zspfj                          1/1     Running    1 (16h ago)  17h
kube-system svclb-traefik-88d3016d-vc66s                           2/2     Running    2 (16h ago)  17h
kube-system traefik-6cd8c7cd89-pxk64                               1/1     Running    1 (16h ago)  17h
ubuntu@cd-server:~$

```

K3s node Ready và Pod hệ thống Running.

6.4. Cấu hình kubectl cho user ubuntu

MÁY / VỊ TRÍ K8s-ArgoCD | Thư mục: /home/ubuntu

```

mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config
chmod 600 ~/.kube/config
kubectl get nodes

```

□ **Ý nghĩa:** Copy kubeconfig để user ubuntu chạy kubectl không cần sudo.

6.5. Cài Argo CD

MÁY / VỊ TRÍ K8s-ArgoCD | Thư mục: /home/ubuntu

```

kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl get pods -n argocd -w

```

□ **Ý nghĩa:** Tạo namespace và triển khai toàn bộ thành phần Argo CD.

```

ubuntu@cd-server:~$ sudo kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION    INTERNAL-IP    EXTERNAL-IP    OS-IMAGE             KERNEL-VERSION    CONTAINER-RUNTIME
cd-server            Ready    control-plane   17h   v1.36.2+k3s1  172.31.45.152    <none>         Ubuntu 24.04.4 LTS   7.0.0-1009-aws (amd64)  containerd://
2.3.2-k3s2
ubuntu@cd-server:~$ sudo kubectl get pods -A
NAMESPACE   NAME                                     READY   STATUS    RESTARTS   AGE
argocd      argocd-application-controller-0        1/1     Running   1 (16h ago)  17h
argocd      argocd-applicationset-controller-85d4d9d4d-4lqv9  0/1     CrashLoopBackOff   70 (109s ago)  17h
argocd      argocd-dex-server-5bf99699b5-nd2hs      1/1     Running   1 (16h ago)  17h
argocd      argocd-notifications-controller-5bf6c68f6d-zxsdv  1/1     Running   1 (16h ago)  17h
argocd      argocd-redis-6548db7b7c-6psp2           1/1     Running   1 (16h ago)  17h
argocd      argocd-repo-server-697876794c-wr112     1/1     Running   1 (16h ago)  17h
argocd      argocd-server-699676944-58hb7          1/1     Running   1 (16h ago)  17h
default     sample-ci-app-85b74d645f-cfrnb          1/1     Running   0           129m
kube-system coredns-5f5694d56b-dsb9f               1/1     Running   1 (16h ago)  17h
kube-system helm-install-traefik-88qk2             0/1     Completed 2 (17h ago)  17h
kube-system helm-install-traefik-crd-5145f  0/1     Completed 0           17h
kube-system local-path-provisioner-58d557dc48-vzk6t  1/1     Running   1 (16h ago)  17h
kube-system metrics-server-7c86f97b8d-zspfj     1/1     Running   1 (16h ago)  17h
kube-system svclb-traefik-88d3016d-vc66s    2/2     Running   2 (16h ago)  17h
kube-system traefik-6cd8c7cd89-pxk64       1/1     Running   1 (16h ago)  17h
ubuntu@cd-server:~$

```

Các Pod Argo CD Running

6.6. Expose Argo CD bằng NodePort

MÁY / VỊ TRÍ K8s-ArgoCD | Thư mục: /home/ubuntu

```

kubectl patch svc argocd-server -n argocd -p '{"spec":{"type":"NodePort"}}'
kubectl get svc argocd-server -n argocd

```

□ **Ý nghĩa:** Đổi Service từ ClusterIP thành NodePort. Kubernetes tự cấp hai NodePort cho HTTP và HTTPS.

Ví dụ lab đã từng nhận HTTPS NodePort 31140. Cần lấy đúng cổng thực tế trong cột PORT(S), không dùng cổng đoán trước.

```

ubuntu@cd-server:~$ sudo kubectl get svc argocd-server -n argocd
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)                                     AGE
argocd-server       NodePort    10.43.15.211   <none>         80:30402/TCP,443:31140/TCP               17h
ubuntu@cd-server:~$

```

Service argocd-server kiểu NodePort

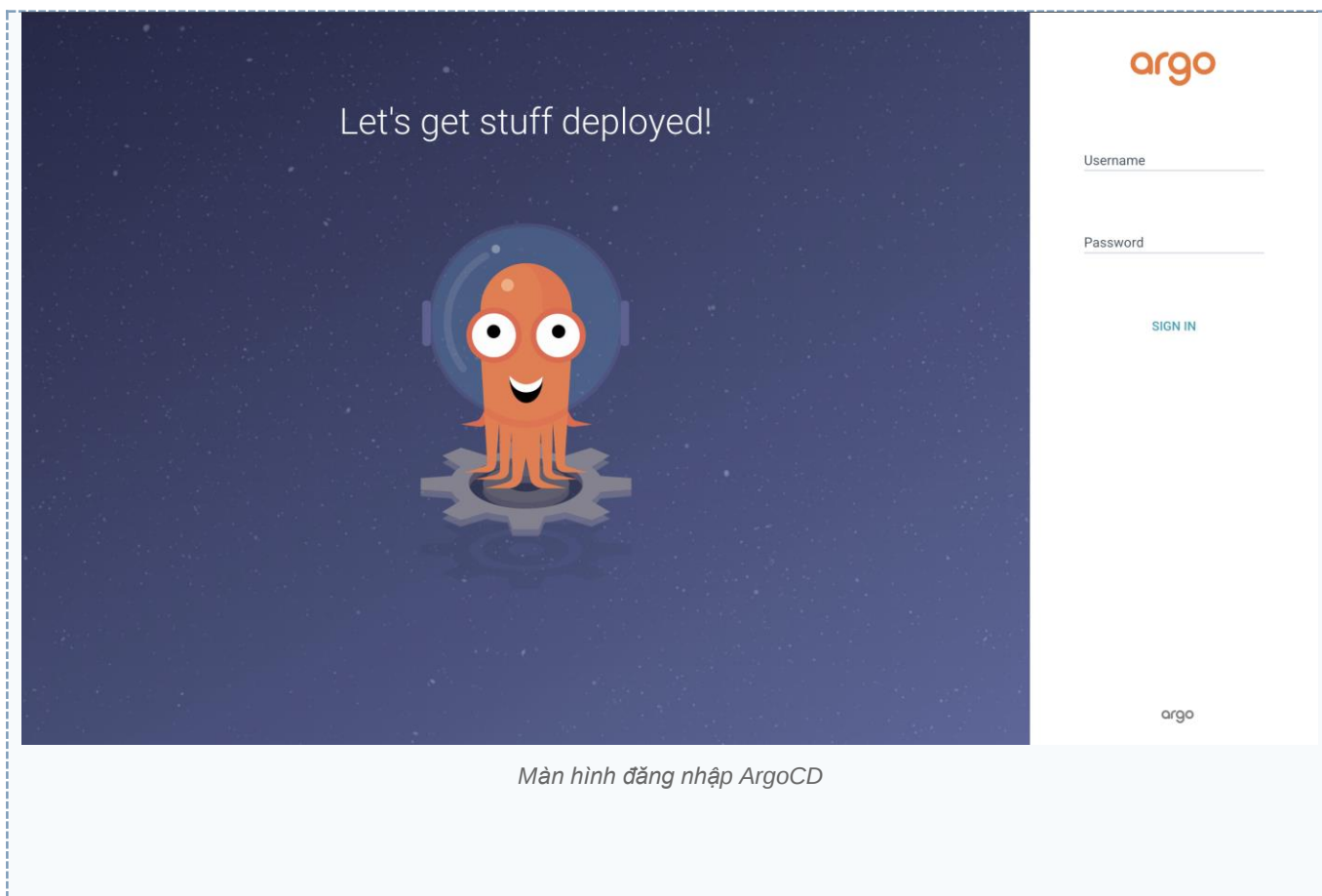
6.7. Đăng nhập Argo CD

```

kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d; echo

```

31. Mở <https://PUBLIC-IP-CD:NODEPORT-HTTPS>.
32. Chấp nhận cảnh báo self-signed certificate trong trình duyệt.
33. Username: admin; Password: kết quả lệnh trên.



7. TẠO KUBERNETES MANIFESTS VÀ CẤU HÌNH ARGO CD APPLICATION

7.1. Tạo thư mục k8s trong repo jenkins-ci-lab

MÁY / VỊ TRÍ

Máy dùng để chỉnh source hoặc GitHub Web Editor | Thư mục: Repo jenkins-ci-lab

```
mkdir -p k8s
```

7.2. deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sample-ci-app
spec:
  replicas: 1
  revisionHistoryLimit: 2
  selector:
    matchLabels:
      app: sample-ci-app
  template:
    metadata:
      labels:
        app: sample-ci-app
    spec:
      containers:
        - name: sample-ci-app
          image: anhkv97/sample-ci-app:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 3000
```

□ **Ý nghĩa:** Deployment duy trì một Pod, giữ tối đa hai ReplicaSet cũ và kéo image Docker Hub theo tag do Jenkins cập nhật.

7.3. service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: sample-ci-service
spec:
  type: NodePort
  selector:
    app: sample-ci-app
  ports:
    - protocol: TCP
      port: 3000
      targetPort: 3000
      nodePort: 30080
```

□ **Ý nghĩa:** Mở cổng 30080 trên node, chuyển request vào port 3000 của Pod Node.js.

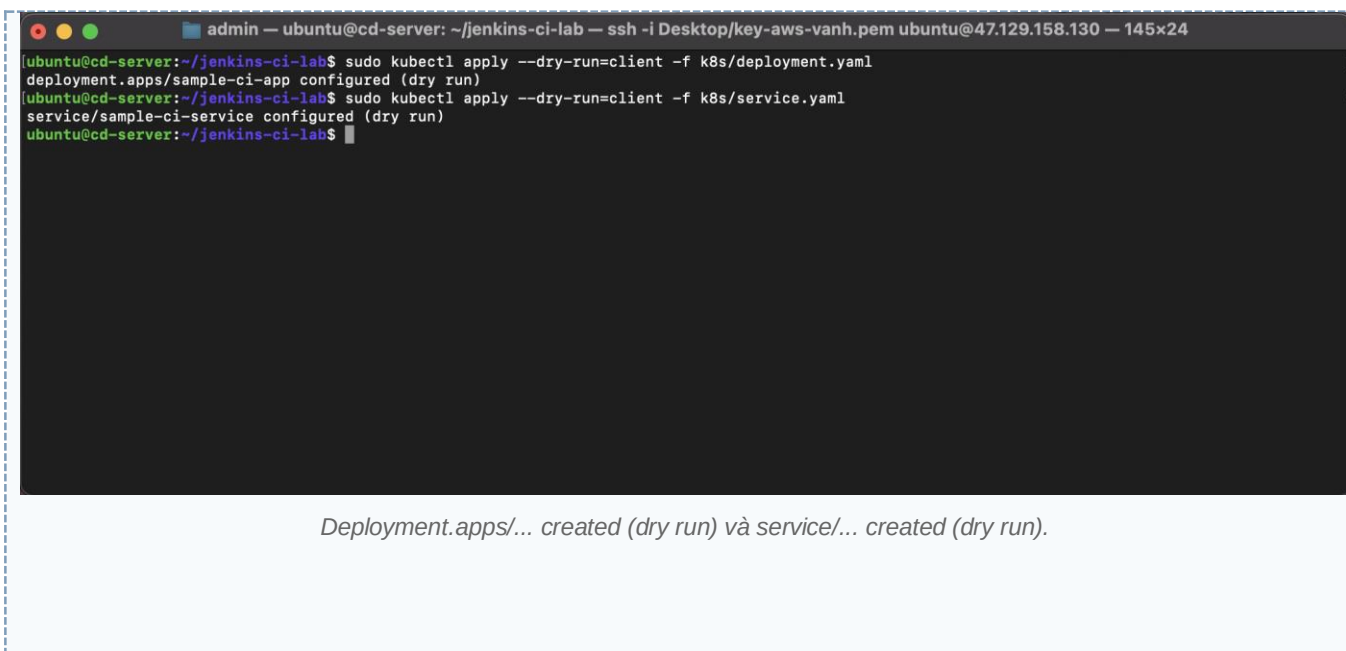
7.4. Kiểm tra cú pháp manifest

MÁY / VỊ TRÍ

K8s-ArgoCD | Thư mục: /home/ubuntu/jenkins-ci-lab hoặc thư mục clone repo

```
kubectl apply --dry-run=client -f k8s/deployment.yaml
kubectl apply --dry-run=client -f k8s/service.yaml
```

□ **Ý nghĩa:** Kiểm tra cú pháp phía client nhưng chưa triển khai thật.



7.5. Tạo Argo CD Application

34. Argo CD → Applications → New App.
35. Application Name: sample-ci-app; Project: default.
36. Sync Policy: Automatic; bật Prune Resources và Self Heal.
37. Repository URL: <https://github.com/anhkv100297/jenkins-ci-lab.git>
38. Revision: main; Path: k8s.
39. Cluster URL: <https://kubernetes.default.svc>; Namespace: default.
40. Create.

Form tạo Argo CD Application

7.6. Kiểm tra triển khai

MÁY / VỊ TRÍ K8s-ArgoCD | Thư mục: /home/ubuntu

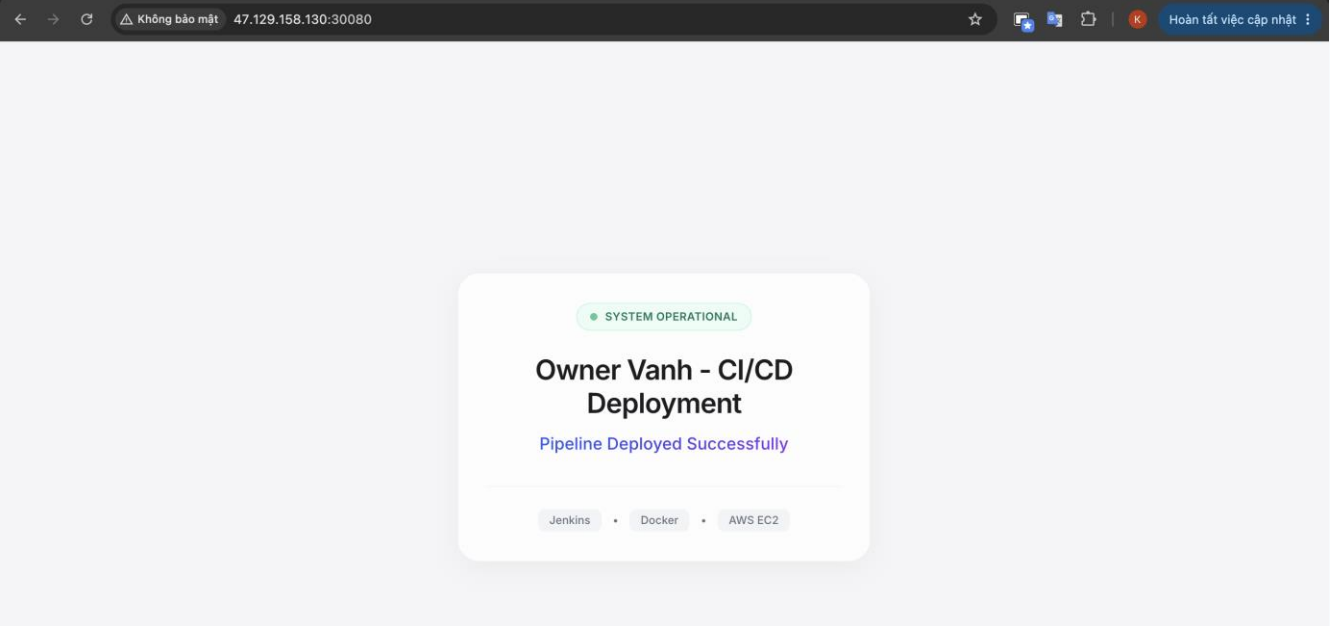
```
kubectl get deployment
kubectl get rs
kubectl get pods
kubectl get svc
```

☐ **Ý nghĩa:** Kiểm tra Deployment, ReplicaSet, Pod và Service được Argo CD tạo.

```
[ubuntu@cd-server:~/jenkins-ci-lab$ sudo kubectl get deployment
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
sample-ci-app       1/1     1             1           5h28m
[ubuntu@cd-server:~/jenkins-ci-lab$ sudo kubectl get rs
NAME                DESIRED   CURRENT   READY   AGE
sample-ci-app-6b45477588   0         0         0       3h7m
sample-ci-app-85b74d645f   1         1         1       175m
[ubuntu@cd-server:~/jenkins-ci-lab$ sudo kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
sample-ci-app-85b74d645f-cfrnb   1/1     Running   0          175m
[ubuntu@cd-server:~/jenkins-ci-lab$ sudo kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.43.0.1    <none>        443/TCP          18h
sample-ci-service   NodePort    10.43.221.64 <none>        3000:30080/TCP   5h29m
ubuntu@cd-server:~/jenkins-ci-lab$
```

Pod 1/1 Running và Service NodePort 30080.

Truy cập ứng dụng: <http://47.129.158.130:30080> (thay bằng Public IP hiện tại của CD Server).



Website CI/CD Deployment hiển thị thành công.

8. KIỂM THỬ TOÀN BỘ LƯỒNG CI/CD GITOPS

8.1. Kịch bản kiểm thử

41. Sửa một dòng hiển thị trong app.js, ví dụ đổi tiêu đề hoặc subtitle.
42. Commit và push lên branch main.

MÁY / VỊ TRÍ

Máy chính source | Thư mục: Thư mục root repo jenkins-ci-lab

```
git add app.js
git commit -m "Test final GitOps pipeline"
git push origin main
```

8.2. Quan sát Jenkins

Webhook gọi Jenkins. Build đầy đủ phải đi qua các bước Checkout, Install dependencies, Test, Build Docker image, Push Docker image, Update Kubernetes manifest và Push manifest to GitHub.

The screenshot displays the Jenkins web interface for a build named 'sample-ci-pipeline' at build number 30. The interface includes a left sidebar with navigation options like Status, Changes, Console Output, and Pipeline Overview. The main area shows a green checkmark indicating a successful build. Key details include the build started by a GitHub push, the revision hash, the repository URL, and the build steps: 'test FULL CICD'. The build took 24 seconds and was completed 2 hours and 59 minutes ago. At the bottom, a message states 'Jenkins build đầy đủ thành công'.

Sau build chính, Jenkins có thể sinh thêm một build rất nhanh với mô tả “Skipped GitOps manifest commit”. Đây là build bảo vệ vòng lặp và là hành vi đúng.

Jenkins

sample-ci-pipeline

#31

Status

Changes

Console Output

Edit Build Information

Delete build '#31'

Polling Log

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

#31 (07:26:47, 3 thg 8, 2026)

Skipped GitOps manifest commit

Started by GitHub push by anhkv100297

Revision: c6e1a12be97c7801f545028d708792ccc7c9f9fc

Repository: <https://github.com/anhkv100297/jenkins-ci-lab.git>

refs/remotes/origin/main

Update image to anhkv97/sample-ci-app:30 [skip ci] c6e1a12

jenkins at 07:26 03/08/2026

Edit description

Keep this build forever

Started 3 hr 0 min ago

Took 4.4 sec

Build thứ hai bị skip an toàn

8.3. Quan sát Docker Hub

Docker Hub xuất hiện tag mới tương ứng BUILD_NUMBER, đồng thời tag latest được cập nhật.

anhkv97

Your personal account

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Upgrade plan

Repositories / sample-ci-app / General

anhkv97/sample-ci-app

Last pushed about 3 hours ago · Repository size: 60.5 MB · ☆0 · ↓390

Add a description

Add a category

General

Tags

Image Management

Collaborators

Webhooks

Settings

Tags

This repository contains 16 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	about 3 hours
30		Image	less than 1 day	about 3 hours
28		Image	less than 1 day	about 3 hours
27		Image	less than 1 day	about 3 hours
26		Image	less than 1 day	about 3 hours

See all

Docker commands

Public view

To push a new tag to this repository:

docker push anhkv97/sample-ci-app:tagname

buildcloud

Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

Go to Docker Build Cloud

Docker Hub có tag Build_Number mới.

8.4. Quan sát Argo CD và Kubernetes

MÁY / VỊ TRÍ

K8s-ArgoCD | Thư mục: /home/ubuntu

```
kubectl get pods -w
```

□ **Ý nghĩa:** Theo dõi Rolling Update: Pod mới ContainerCreating/Running và Pod cũ Terminating.

```
[ubuntu@cd-server:~/jenkins-ci-lab$ sudo kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
sample-ci-app-85b74d645f-cfrnb      1/1     Running   0          3h1m
```

Rolling Update trên CLI.



Argo CD Synced và Healthy.

8.5. Xác nhận kết quả cuối

43. Refresh website tại <http://PUBLIC-IP-CD:30080>.
44. Xác nhận nội dung giao diện đã thay đổi theo app.js mới.
45. Không SSH thủ công để docker build/docker push; toàn bộ phải tự động.

□ Tiêu chí hoàn thành bài lab

Một lần git push phải kích hoạt Jenkins, tạo image mới trên Docker Hub, cập nhật manifest GitHub, Argo CD tự Sync và website hiển thị phiên bản mới.

9. DỌN DỆP, VẬN HÀNH VÀ XỬ LÝ LỖI THƯỜNG GẶP

9.1. Dọn ReplicaSet cũ

MÁY / VỊ TRÍ

K8s-ArgoCD | Thư mục: /home/ubuntu

```
sudo kubectl get rs --no-headers | awk '$2==0 {print $1}' | xargs -r sudo kubectl delete rs
```

□ **Ý nghĩa:** Xóa toàn bộ ReplicaSet có DESIRED=0; giữ nguyên ReplicaSet đang chạy DESIRED=1.

```
kubectl get rs
kubectl get pods
```

```

ubuntu@cd-server:~/jenkins-ci-lab$ sudo kubectl get rs
NAME                                DESIRED    CURRENT    READY    AGE
sample-ci-app-6b45477588           0          0          0        3h16m
sample-ci-app-85b74d645f           1          1          1        3h4m
ubuntu@cd-server:~/jenkins-ci-lab$ sudo kubectl get pods
NAME                                READY      STATUS    RESTARTS   AGE
sample-ci-app-85b74d645f-cfrnb      1/1       Running   0          3h4m
ubuntu@cd-server:~/jenkins-ci-lab$

```

Chỉ còn một ReplicaSet và Một Pod Running.

9.2. Dọn Docker image trên CI-Lab

MÁY / VỊ TRÍ CI-Lab | Thư mục: /home/ubuntu

```

docker image prune -f
docker system df

```

□ **Ý nghĩa:** Xóa dangling images và kiểm tra dung lượng Docker. Jenkinsfile cũng tự chạy image prune sau mỗi build.

9.3. Lỗi ImagePullBackOff / ErrImagePull

```

kubectl describe pod TEN_POD

```

Nguyên nhân thường gặp: sai Docker Hub username, repository chưa tồn tại, tag chưa được push, repository private nhưng chưa cấu hình imagePullSecret.

Kiểm tra image trong manifest:

```

kubectl get deployment sample-ci-app -o jsonpath='{.spec.template.spec.containers[0].image}'; echo

```

9.4. Không truy cập được NodePort

- Pod phải ở trạng thái 1/1 Running; nếu Pod lỗi thì Service không có backend.
- Kiểm tra đúng NodePort bằng kubectl get svc; không dùng cổng đoán trước.
- Mở đúng port trong AWS Security Group.
- Với Argo CD, HTTPS NodePort phải truy cập bằng https://.

9.5. Git push bị non-fast-forward

```

git pull --rebase origin main
git push origin main

```

□ **Ý nghĩa:** Lấy commit mới từ remote và đặt commit local lên trên, tránh merge commit không cần thiết.

□ **Không dùng git push -f**

Force push có thể ghi đè commit do Jenkins tạo và làm hỏng lịch sử GitOps.

9.6. Jenkins chạy vòng lặp

- Đảm bảo commit Jenkins tạo có [skip ci].
- Jenkinsfile phải return ngay khi commit message chứa [skip ci] hoặc chỉ thay đổi k8s/.
- Có thể thấy một build thứ hai bị skipped; đây không phải vòng lặp.
- Nếu build tiếp tục tăng liên tục và đều build image, tạm tắt Webhook Active rồi sửa Jenkinsfile.

9.7. Argo CD ApplicationSet Controller CrashLoopBackOff

Nếu argocd-server vẫn Running, Web UI có thể vẫn truy cập được. Kiểm tra log Pod bị lỗi và tài nguyên RAM. Máy t3.small có giới hạn 2 GiB nên không nên chạy thêm workload nặng.

```
kubectl logs -n argocd POD_NAME --previous
free -h
df -h
```

9.8. Dừng máy để tiết kiệm chi phí

Khi không học, có thể Stop cả hai EC2. Elastic IP của CI-Lab vẫn giữ nguyên khi Start lại. CD Server có thể nhận IP Public mới; cần dùng IP mới để truy cập Argo CD và ứng dụng.

❑ Chi phí Elastic IP

Kiểm tra chính sách giá AWS hiện hành. Elastic IP có thể phát sinh phí tùy trạng thái sử dụng và chính sách tại thời điểm triển khai.

10. PHỤ LỤC: FILE CẤU HÌNH VÀ BẢNG LỆNH TRA CỨU

10.1. Bảng lệnh CI-Lab

Lệnh	Mô tả
docker compose up -d	Khởi chạy Jenkins container
docker compose ps	Kiểm tra trạng thái Jenkins
docker logs -f jenkins	Theo dõi log Jenkins
docker exec -it jenkins bash	Vào shell Jenkins container
docker images	Xem Docker images trên host
docker system df	Xem dung lượng Docker
docker image prune -f	Dọn dangling images

10.2. Bảng lệnh K3s/Kubernetes

Lệnh	Mô tả
kubectl get nodes -o wide	Xem node và trạng thái Ready
kubectl get pods -A	Xem Pod tất cả namespace
kubectl get pods -w	Theo dõi Pod theo thời gian thực
kubectl get svc	Xem Service và NodePort
kubectl get rs	Xem ReplicaSet và revision
kubectl describe pod <pod>	Xem chi tiết và Events của Pod

Lệnh	Mô tả
kubectl logs <pod>	Xem log container
kubectl rollout status deployment/sample-ci-app	Kiểm tra rollout
kubectl rollout history deployment/sample-ci-app	Xem lịch sử rollout
kubectl delete pod -l app=sample-ci-app	Xóa Pod để Deployment tạo lại

10.3. Bảng lệnh Argo CD

Lệnh	Mô tả
kubectl get pods -n argocd	Kiểm tra Pod Argo CD
kubectl get svc -n argocd	Xem Service và NodePort Argo CD
kubectl logs -n argocd <pod>	Xem log thành phần Argo CD
kubectl get applications -n argocd	Xem Argo CD Applications bằng kubectl

□ Kết luận

Hệ thống hoàn chỉnh sử dụng một GitHub repository duy nhất. Jenkins thực hiện CI và cập nhật image tag trong manifest; Argo CD thực hiện CD theo mô hình pull-based GitOps; Kubernetes đảm bảo self-healing và Rolling Update.